Função é um bloco de código reutilizável

exemplo jogo da velha

```
retorno_tipo nome(tipo param1, tipo param2) {

// corpo da função

return valor;
}
```

imprimindo na tela

```
void mensagem() {
     printf("Bom dia!\n");
}

int main() {
     mensagem();
return 0;
}
```

void -> não retorna nada

RETORNANDO VALORES

```
int soma(int a, int b) {
     return a + b;
}


int main(){
     int resultado = soma(3, 5);
     printf("Resultado: %d", resultado);
return 0;
}
```

IMPRIMINDO DIRETO

```
void soma(int a, int b) {
     int resultado;
     resultado = a + b;
     printf(”Resultado: %d”, resultado);
}


int main(){
     soma(3, 5);
return 0;
}
```

a e b só existem dentro do escopo

Uma função recursiva é uma função que chama a si mesma

Dois elementos:

Caso base : condição de parada

Chamada recursiva : a função chamando a si mesma com um problema menor

```
void contagem(int n) {
      if (n == 0) { // caso base
         return;
      }
      printf("%d", n);
      contagem(n - 1); // chamada recursiva
}


int main() {
      contagem(5);
return 0;
}
```

A função imprime n

Chama a si mesma com n-1

Para quando n == 0

ex clássico fatorial

```
int fatorial(int n) {
       if (n == 0) { // caso base
          return 1;
       }
       return n * fatorial(n - 1); // chamada recursiva
}
```

*

```
int x = 10;
int *px = &x;
```

```
printf("Valor de x: %d\n", x);
printf("Endereco de x: %p\n", &x);
printf("Valor do ponteiro px: %p\n", px);
printf("Valor apontado pelo ponteiro px: %d\n", *px);
```

adicionando py

```
int x = 10;
int *px = &x;

int *py = &x;
*py = *py + 1;

printf("Valor de x: %d\n", x);
printf("Endereco de x: %p\n", &x);
printf("Valor do ponteiro px: %p\n", px);
printf("Valor apontado pelo ponteiro px: %d\n", *px);

printf("Valor do ponteiro py: %p\n", py);
printf("Valor apontado pelo ponteiro py: %d\n", *py);
```

vetor

```
int v[3] = {10, 20, 30};
int *p = v;


printf("%d\n", *p); // 10
printf("%d\n", *(p+1)); // 20
printf("%d\n", *(p+2)); // 30
```

analogia chrome

qual o problema da função?

```
void troca(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}
```

passando por referência

```
void troca(int *a, int *b) {
      int temp = *a;
      *a = *b;
      *b = temp;
}

int main() {
      int x = 5, y = 10;
      troca(&x, &y);
      printf("x = %d, y = %d", x, y);
      return 0;
}
```

DICAS

&x envia o endereço

int *a recebe o endereço

*a acessa o valor original

referencia + retorno

```
int somaEProduto(int a, int b, int *produto) {
    *produto = a * b;   // altera a variável externa
    return a + b;       // retorna outro resultado
}

int main() {
    int x = 3, y = 4;
    int prod;
    int soma;

    soma = somaEProduto(x, y, &prod);

    printf("Soma: %d\n", soma);
    printf("Produto: %d\n", prod);

    return 0;
}
```

Escreva uma função recursiva que receba um número inteiro positivo N e imprima, em ordem decrescente, todos os números entre 1 e N, inclusive, que são divisíveis por 3. Indique também como será a chamada para a função no programa principal. Não é necessário fazer o programa principal, apenas como seria a chamada, dado um número inteiro como int N.

```
void imprimeDiv3(int n) {
    if (n == 0) {           // caso base
        return;
    }

    if (n % 3 == 0) {
        printf("%d\n", n);
    }

    imprimeDiv3(n - 1);    // chamada recursiva
}
```

Faça um programa que leia um vetor de n inteiros definidos pelo usuário (n ≤ 50).
Além disso, crie uma função que receba este vetor já preenchido, o número de elementos deste vetor, e quaisquer dados que sejam necessários, e retorne para a função principal a quantidade de valores maiores que 100, a quantidade de valores maiores que 1000, e a soma dos valores maiores que 1000.
Os valores devem ser inseridos pelo usuário na função principal, e os resultados (quantidade de valores maiores que 100, quantidade de valores maiores que 1000 e a soma dos valores maiores que 1000) devem ser impressos na função principal.

```c
void analisaVetor(int v[], int n,
                  int *qtdMaior100,
                  int *qtdMaior1000,
                  int *somaMaior1000) {

    *qtdMaior100 = 0;
    *qtdMaior1000 = 0;
    *somaMaior1000 = 0;

    for (int i = 0; i < n; i++) {
        if (v[i] > 100) {
            (*qtdMaior100)++;
        }
        if (v[i] > 1000) {
            (*qtdMaior1000)++;
            *somaMaior1000 += v[i];
        }
    }
}
```

```c
int main() {
    int n;
    int v[50];

    int qtdMaior100;
    int qtdMaior1000;
    int somaMaior1000;

    printf("Digite o tamanho do vetor (n <= 50): ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Digite o elemento %d: ", i + 1);
        scanf("%d", &v[i]);
    }

    analisaVetor(v, n, &qtdMaior100, &qtdMaior1000, &somaMaior1000);

    printf("\nQuantidade de valores maiores que 100: %d\n", qtdMaior100);
    printf("Quantidade de valores maiores que 1000: %d\n", qtdMaior1000);
    printf("Soma dos valores maiores que 1000: %d\n", somaMaior1000);

    return 0;
}
```

```
n=5
84
2160
762
1300
1540
```

Considere a estrutura Aluno definida por typedef struct aluno {int matricula; float n1, n2;} Aluno;

Faça uma função VOID que receba o vetor de 20 alunos, e calcule a média da primeira nota entre todos os alunos, e a média da segunda nota entre todos os alunos e retorne esses valores por parâmetro, que também devem ser vistos no programa principal.

Utilize ponteiros quando necessário.

Faça o programa principal que aloque dinamicamente memória para armazenar as informações dos 20 alunos, leia estes dados a partir do usuário, chame a função e mostre na tela as duas médias calculadas.

```c
typedef struct aluno {
    int matricula;
    float n1, n2;
} Aluno;

void calculaMedias(Aluno *alunos, int qtd, float *mediaN1, float *mediaN2) {
    float somaN1 = 0.0, somaN2 = 0.0;

    for (int i = 0; i < qtd; i++) {
        somaN1 += alunos[i].n1;
        somaN2 += alunos[i].n2;
    }

    *mediaN1 = somaN1 / qtd;
    *mediaN2 = somaN2 / qtd;
}
```

```c
int main() {
    Aluno *alunos;
    int qtd = 5;

    float mediaN1, mediaN2;

    /* Alocação dinâmica */
    alunos = (Aluno *) malloc(qtd * sizeof(Aluno));

    /* Leitura dos dados */
    for (int i = 0; i < qtd; i++) {
        printf("\nAluno %d\n", i + 1);
        printf("Matricula: ");
        scanf("%d", &alunos[i].matricula);
        printf("Nota 1: ");
        scanf("%f", &alunos[i].n1);

        printf("Nota 2: ");
        scanf("%f", &alunos[i].n2);
    }

    /* Chamada da função */
    calculaMedias(alunos, qtd, &mediaN1, &mediaN2);

    /* Exibição dos resultados */
    printf("\nMedia da N1: %.2f\n", mediaN1);
    printf("Media da N2: %.2f\n", mediaN2);

    /* Liberação da memória */
    free(alunos);

    return 0;
}
```